Project Specification: Enclave Management System

A High-Performance PWA using Rust (Yew + Poem)

System Architecture Design

September 27, 2026

Contents

1 Executive Summary **2**

2 Technological Stack **2**

3 Functional Requirements **2**
- 3.1 Identity and Access Management (IAM) . . . 2
- 3.2 Mess and Attendance Module . . . 2
- 3.3 Communication and Grievance . . . 3

4 System Architecture and Security **3**
- 4.1 Strict Middleware Security . . . 3
- 4.2 Data Model (Simplified) . . . 3

5 Implementation Roadmap **3**

6 Non-Functional Requirements **3**

7 Phase 1: Workspace & Shared Data Models **4**

8 Phase 2: Backend Security & Database Logic **4**

9 Phase 3: The Auth Bridge (Cloudflare Worker) **4**

10 Phase 4: Frontend PWA Onboarding UI **5**

11 Phase 5: Concurrency Automation Features **5**

12 Phase 6: Admin Panel & Redressal **5**

1 Executive Summary

The Enclave Management System is a unified Progressive Web Application (PWA) designed to streamline residential operations. Built with a focus on safety and performance using the Rust programming language, the system manages mess attendance, committee communications, grievance redressal, and automated document generation. Key highlights include a zero-trust administrative verification gate and an OTP-less authentication system.

2 Technological Stack

The application utilizes a modern, type-safe stack to ensure memory safety and high concurrency.

- **Frontend:** `Yew.rs` (WebAssembly framework) with `TailwindCSS`.
- **Backend:** `Poem` (Web framework) with `Poem-OpenAPI` for type-safe REST contracts.
- **Database:** `PostgreSQL` for persistent storage and `Redis` for high-speed session/concurrency management.
- **Infrastructure:** `Cloudflare Workers` for Edge-computing and WhatsApp API integration.
- **Database Access:** `SQLx` (Asynchronous, compile-time checked SQL queries).
- **Document Generation:** `Typst` or `genpdf` for automated PDF rendering.

3 Functional Requirements

3.1 Identity and Access Management (IAM)

1. **OTP-less Onboarding:** Users register via a phone number. A Cloudflare Worker triggers a WhatsApp message containing a cryptographically secure "Magic Link."
2. **Admin Verification Gate:** All new accounts default to a `Pending` state. Users are restricted from all functional modules until an administrator manually verifies their residency status.
3. **Biometric Daily Login:** To minimize WhatsApp API costs and latency, the system utilizes **WebAuthn (Passkeys)**. After the first login, users can authenticate via FaceID/TouchID.

3.2 Mess and Attendance Module

1. **Daily Tracking:** A simplified interface for Breakfast, Lunch, and Dinner check-ins.
2. **Concurrency Handling:** High-volume check-ins (e.g., during peak mess hours) are buffered in Redis before being committed to PostgreSQL to prevent database lock contention.
3. **Mess Concession:** A digital form allowing students to request leave. Upon submission, the system validates the dates and generates a formal PDF letter for the mess committee.

3.3 Communication and Grievance

1. **Committee Bulletin:** A multi-channel feed where authorized committee members (Sports, Mess, Cultural) post updates.
2. **Common Calendar:** A unified schedule for enclave-wide events with filtering capabilities.
3. **Grievance Redressal:** A ticketing system with status updates (Open, In-Progress, Resolved) and an integrated FAQ section for instant self-help.

4 System Architecture and Security

4.1 Strict Middleware Security

To defend against unauthorized access and common web vulnerabilities:

- **JWT Validation:** Every request must include a JWT in the Authorization header. The backend strictly checks the `iss` (issuer), `aud` (audience), and `exp` (expiry).
- **SQL Injection Prevention:** By using `SQLx` prepared statements, user input is never concatenated into SQL queries.
- **Role-Based Access Control (RBAC):** Specific endpoints (e.g., `/admin/approve`) are protected by middleware that verifies the `is_admin` flag within the JWT claims.

4.2 Data Model (Simplified)

```
CREATE TYPE account_status AS ENUM ('pending', 'active', 'suspended');

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    whatsapp_no TEXT UNIQUE,
    status account_status DEFAULT 'pending',
    is_admin BOOLEAN DEFAULT FALSE
);

CREATE TABLE attendance (
    user_id UUID REFERENCES users(id),
    meal_type VARCHAR(10),
    date DATE,
    PRIMARY KEY (user_id, meal_type, date)
);
```

Listing 1: Core Database Entities

5 Implementation Roadmap

6 Non-Functional Requirements

- **Latency:** Attendance marking must respond in < 200ms.
- **Availability:** Offline-first capability via PWA Service Workers for reading bulletins.
- **Scalability:** Stateless backend design to allow horizontal scaling via Docker containers.

Phase	Duration	Deliverables
Phase 1	Week 1	Cargo Workspace, DB Schema, Poem API Boilerplate
Phase 2	Week 2	WhatsApp Worker, Admin Approval UI, JWT Middleware
Phase 3	Week 3	Attendance System, Redis Integration, PWA Manifest
Phase 4	Week 4	PDF Generation, Bulletin Feed, Grievance Module
Phase 5	Week 5	Load Testing, Security Hardening, Final Deployment

Table 1: Project Roadmap

Instructions for Use

To build this application using an AI, **do not** paste the entire requirement at once. Follow the phases below sequentially. Ensure the AI confirms completion of each phase before moving to the next.

0: The System Context (Run First) "I am building a high-performance **Enclave Management System**. **Stack:** Rust Workspace with Frontend (Yew + WASM) and Backend (Poem + Poem-OpenAPI). **Database:** PostgreSQL with SQLx (Async). **Cache:** Redis for high-concurrency attendance. **Auth:** OTP-less via Cloudflare Workers + WhatsApp Business API. **Security:** Strict JWT, Admin-only verification gate (Pending/Active status), and 100% SQL Injection prevention via SQLx prepared statements.

Act as a Senior Rust Architect. We will build this modularly. Please acknowledge the architecture and stack. Do not write code yet."

7 Phase 1: Workspace & Shared Data Models

1: Data Contracts "Initialize a Cargo Workspace with crates: `frontend`, `backend`, and `common`. In the `common` crate: 1. Define a `UserStatus` enum (Pending, Active, Suspended). 2. Define a `User` struct and `Attendance` struct using `serde`. 3. Use the `validator` crate to define a `RegistrationRequest` struct with regex for phone numbers and roll numbers. 4. Provide the PostgreSQL schema migration for a `users` table with a `status` column defaulting to 'pending'."

8 Phase 2: Backend Security & Database Logic

2: Poem API & Middleware "In the `backend` crate: 1. Setup `Poem-OpenAPI` to generate a Swagger UI. 2. Implement an `AuthMiddleware` that validates JWTs strictly (check iss, aud, exp). 3. Implement a `VerificationGuard` middleware: it must check the Database/Redis to ensure the user's status is 'Active'. If 'Pending', return a 403 Forbidden. 4. Use `sqlx::query!` macros for all DB operations to ensure compile-time SQL safety and prevent SQL injection."

9 Phase 3: The Auth Bridge (Cloudflare Worker)

3: OTP-less Bridge "Write a Cloudflare Worker script (Typescript or Rust) to handle WhatsApp Authentication: 1. Receive a login request from the Poem backend. 2. Generate a secure, short-lived nonce and store it in Cloudflare KV. 3. Trigger a WhatsApp Business API message with a 'Magic Link' containing the token. 4. Define the callback route in the Poem backend that verifies this token and issues the final JWT."

10 Phase 4: Frontend PWA Onboarding UI

4: Yew Frontend Foundation "In the `frontend` crate: 1. Setup `yew-router` and `yewdux` for state management. 2. Create a 'Login' page and a 'Waiting for Approval' lobby. 3. The 'Lobby' must poll the backend to check if the Admin has changed the user status to 'Active'. 4. Setup `Trunk.toml` and a `manifest.json` for PWA installation and offline caching of the FAQ/Bulletin."

11 Phase 5: Concurrency Automation Features

5: Attendance & Concession Logic "Implement the core feature logic: 1. **Attendance:** On the backend, use Redis to buffer daily check-ins for Bkf/Lun/Din to handle 8:00 AM traffic spikes. 2. **Mess Concession:** Create a Yew form. On submission, the backend should use the `genpdf` crate to render a formal letter including the student's details and dates. 3. Ensure the backend validates that no attendance was marked during the requested concession dates."

12 Phase 6: Admin Panel & Redressal

6: Management Dashboard "Build the Admin interface: 1. Create a route `/admin/verify` that lists users with 'Pending' status. 2. Add functionality for an Admin to click 'Approve', which updates the DB and pushes a notification to the student. 3. Implement the Grievance section where students post tickets and admins update status (Open/Resolved)."